MODULE BONUS

Analyse de données avec Python

Pour aller plus loin
Gros volumes, applications, exécutables et Python intermédiaire

Si le temps le permet, en complément des 21 heures du programme

Au programme, selon le temps et les envies

1	2	3	4	5	6
Gros volumes	**Streamlit**	**Interfaces**	**Exécutable**	**Ligne de commande**	**Python**
Polars, DuckDB	TP bonus A	tkinter, Qt	TP bonus B	Rich, Typer	dataclasses, décorateurs...

Chaque partie est indépendante : on choisit ce qui sert le plus au quotidien des participants.

Pour tester les exemples

```
uv init bonus && cd bonus
uv add pandas pyarrow polars duckdb streamlit plotly rich typer
```

01

Données volumineuses

Polars | DuckDB | Spark

1 Les limites de pandas

Toutes les données **en mémoire**
La plupart des calculs sur **un seul cœur** du processeur

- Pas d'optimisation globale : chaque opération s'exécute dans l'ordre écrit

Tant que les données tiennent confortablement en mémoire, pandas suffit. Au-delà de quelques millions de lignes, Polars et DuckDB prennent le relais, sans changer de langage.

1 Polars : des expressions

```
import polars as pl

resultat = (
    ventes
    .join(produits, on="id_produit", how="left")
    .with_columns(
        (pl.col("prix_unitaire") * pl.col("quantite"))
        .alias("montant"))
    .group_by("categorie")
    .agg(
        pl.col("montant").sum().round(2).alias("ca"),
        pl.len().alias("nb_commandes"),
    )
    .sort("ca", descending=True)
)
```

Écrit en Rust, utilise **tous les cœurs**
On décrit les calculs avec des **expressions** `pl.col(...)`
Pas d'index, `null` pour les valeurs manquantes
`df.to_pandas()` et `pl.from_pandas(df)` pour passer de l'un à l'autre

1 De pandas à Polars

pandas	Polars
`df[df["x"] > 10]`	`df.filter(pl.col("x") > 10)`
`df[["a", "b"]]`	`df.select("a", "b")`
`df["c"] = df["a"] * 2`	`df.with_columns((pl.col("a") * 2).alias("c"))`
`df.groupby("k").agg(...)`	`df.group_by("k").agg(...)`
`df.merge(autre, on="k")`	`df.join(autre, on="k")`
`df.sort_values("x")`	`df.sort("x")`

1 Le mode paresseux

```
ca_idf = (
    pl.scan_parquet("ventes.parquet")
    .filter(pl.col("code_region") == "IDF")
    .group_by(pl.col("date").dt.month()
              .alias("mois"))
    .agg((pl.col("prix_unitaire")
          * pl.col("quantite")).sum()
         .alias("ca"))
    .sort("mois")
    .collect()
)
```

`scan_csv`, `scan_parquet` : rien n'est lu tout de suite
Polars construit un **plan**, l'optimise, puis l'exécute au `collect()`

- Il ne lit que les colonnes et les lignes nécessaires

1 Un piège avec plusieurs fichiers

Erreur

```
pl.read_csv("ressources/ventes/*.csv")
# ComputeError: schema names differ
```

Le fichier de mai a ses colonnes dans un autre ordre. pandas l'acceptait, Polars non.

Solution

```
fichiers = sorted(Path("ressources/ventes")
                  .glob("*.csv"))

ventes = pl.concat(
    [pl.read_csv(f, try_parse_dates=True)
     for f in fichiers],
    how="diagonal",
)
```

`how="diagonal"` aligne les colonnes par leur nom.

1 DuckDB : du SQL sur des fichiers

```
import duckdb

duckdb.sql("""
  SELECT p.categorie,
         round(sum(v.prix_unitaire
                   * v.quantite), 2) AS ca,
         count(*) AS nb_commandes
  FROM read_csv('ressources/ventes/*.csv',
                union_by_name = true) AS v
  LEFT JOIN 'ressources/produits.csv' AS p
    USING (id_produit)
  GROUP BY p.categorie
  ORDER BY ca DESC
""")
```

Une base analytique **sans serveur**, installée comme une bibliothèque
Du SQL directement sur des **CSV** et des **Parquet**, avec `*`
`union_by_name = true`: sans lui, les dates des fichiers du TP3 restent du texte

1 DuckDB avec pandas et Polars

```
resultat = duckdb.sql("SELECT ...")
resultat.df()      # DataFrame pandas
resultat.pl()      # DataFrame Polars

produits_pd = pd.read_csv("ressources/produits.csv")
duckdb.sql("SELECT count(*) FROM produits_pd")          # un DataFrame interrogé par son nom

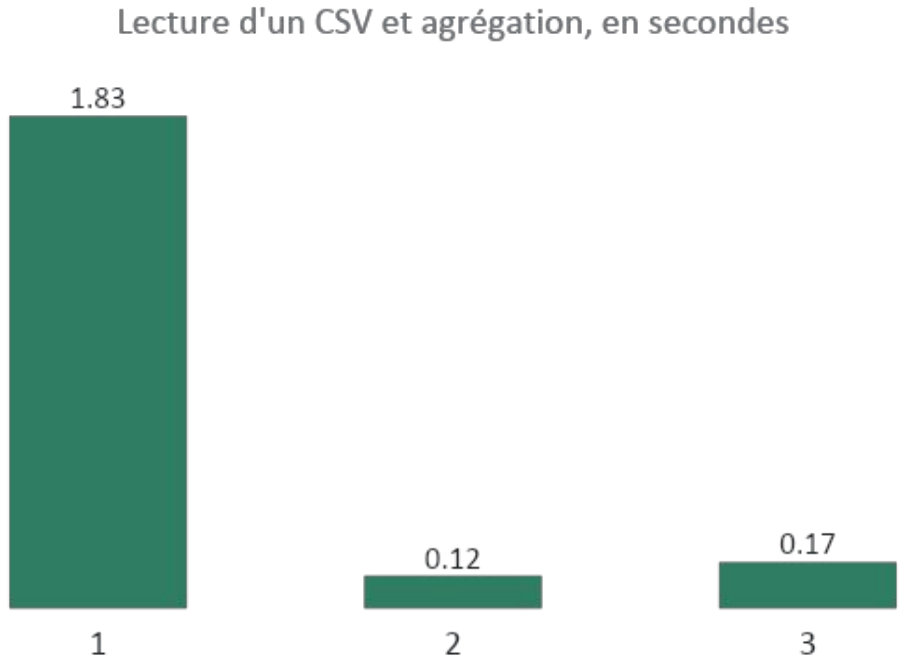
duckdb.sql("COPY (SELECT ...) TO 'outputs/ca.parquet' (FORMAT parquet)")
```

DuckDB peut interroger de gros fichiers sans les charger entièrement en mémoire.

1 Ordres de grandeur : 20 millions de lignes

Source	pandas	Polars	DuckDB
Parquet, 172 Mo	0,34 s	0,06 s	0,02 s
CSV, 448 Mo	1,83 s	0,12 s	0,17 s

Somme et moyenne des montants par région, sur un ordinateur portable récent. Les chiffres dépendent de la machine, l'écart reste.



1 Quel outil choisir ?

Outil	Quand
pandas	les données tiennent en mémoire, l'écosystème compte (seaborn, statsmodels...)
Polars	de gros volumes, une syntaxe proche de pandas, des traitements en Python
DuckDB	de gros fichiers à interroger, une équipe à l'aise en SQL
Spark	des données réparties sur plusieurs machines : une autre échelle, une infrastructure dédiée

02

Streamlit

Une application en un script | Widgets | Cache

2 Une application web en un script

```
import streamlit as st

@st.cache_data
def charger_donnees():
    return pd.read_parquet(
        "ressources/ventes_consolidees.parquet")

ventes = charger_donnees()

st.title("Tableau de bord des ventes 2025")
regions = st.sidebar.multiselect(
    "Régions", sorted(ventes["region"].unique()))
selection = (ventes[ventes["region"].isin(regions)]
             if regions else ventes)

st.metric("Chiffre d'affaires",
          f"{selection['montant'].sum():,.0f} €")
st.plotly_chart(px.histogram(selection, x="montant"))
```

Terminal

```
uv run streamlit run app.py
```

Le script est **réexécuté de haut en bas** à chaque interaction, avec les nouvelles valeurs des filtres.

Pas de HTML ni de JavaScript : que du Python.

2 Les éléments courants

Élément	Rôle	Élément	Rôle
`st.title`, `st.markdown`	textes	`st.metric`	un chiffre clé
`st.sidebar.xxx`	sidebar	`st.columns`	côte à côte
`st.selectbox`	une valeur	`st.plotly_chart`	graphique plotly
`st.multiselect`	plusieurs valeurs	`st.pyplot`	figure matplotlib
`st.select_slider`	un intervalle	`st.dataframe`	tableau triable
`st.tabs`	onglets	`st.download_button`	téléchargement

2 Le cache et le partage

Ne lire qu'une fois

```
@st.cache_data
def charger_donnees():
    return pd.read_parquet(...)
```

Sans cache, chaque clic relirait les fichiers, puisque tout le script est réexécuté.

Partager l'application

Elle tourne sur la machine qui lance `streamlit run`

- Pour d'autres : un serveur interne ou un service d'hébergement
- Attention : les données deviennent accessibles à qui ouvre l'application

TP bonus A : mini tableau de bord Streamlit

Dans le dossier exercices/tpbonus-a-streamlit

1

Filtrer

Régions, période et catégories dans la sidebar.

2

Afficher

Chiffres clés, évolution mensuelle par région, objectifs et meilleurs produits.

3

Livrer

Une application qui se lance avec `uv run streamlit run app.py`.

```
uv sync
uv run streamlit run app.py
```

03

Interfaces graphiques de bureau

tkinter | PyQt et PySide

3 tkinter : une fenêtre sans rien installer

```
import tkinter as tk
from tkinter import filedialog

def choisir_fichier():
    chemin = filedialog.askopenfilename(
        filetypes=[("Fichiers CSV", "*.csv")])
    if chemin:
        ventes = pd.read_csv(chemin)
        total = (ventes["prix_unitaire"]
                 * ventes["quantite"]).sum()
        resultat.set(f"{len(ventes)} ventes, {total:,.0f} €")

fenetre = tk.Tk()
fenetre.title("Ventes")
resultat = tk.StringVar(value="Aucun fichier chargé")
tk.Button(fenetre, text="Choisir un fichier",
          command=choisir_fichier).pack(padx=20, pady=10)
tk.Label(fenetre, textvariable=resultat).pack(pady=10)
fenetre.mainloop()
```

- Inclus dans la bibliothèque standard

Des éléments : `Button`, `Label`, `Entry`
Placés avec `pack` ou `grid`
Une fonction par action : `command=`
`mainloop` attend les actions de l'utilisateur

Suffisant pour un petit utilitaire : choisir un fichier, lancer un traitement, afficher un résultat.

3 Pour aller plus loin : Qt

	tkinter	PySide6	PyQt6
Installation	incluse dans Python	`uv add pyside6`	`uv add pyqt6`
Richesse	éléments de base	tableaux, onglets, graphiques intégrés	identique à PySide6
Apparence	simple	moderne, native	moderne, native
Licence	celle de Python	LGPL	GPL ou commerciale

04

Empaqueter en exécutable

PyInstaller | Un fichier ou un dossier | Points d'attention

4 PyInstaller

lancer_ventes.py

```
import sys

from ventes.cli import main

sys.exit(main())
```

Terminal

```
uv add --dev pyinstaller
uv run pyinstaller --onefile --name ventes
lancer_ventes.py

./dist/ventes --config config.yaml
```

- Rassemble le script, l'interpréteur Python et les bibliothèques

Un `.exe` sous Windows, un binaire sous macOS ou Linux
L'utilisateur n'a **pas besoin de Python**
Résultat dans `dist/` ; `build/` et `.spec` sont des fichiers de travail

4 Un fichier ou un dossier ?

Option	Résultat	Démarrage
`--onefile`	un seul fichier, facile à envoyer	lent : tout est décompressé à chaque lancement
`--onedir`	un dossier avec les bibliothèques	rapide après le premier lancement

La commande ventes, avec pandas

15 s

`--onefile`, 62 Mo

0,2 s

`--onedir`, 175 Mo

Temps d'affichage de l'aide, mesuré sur macOS, hors premier lancement.

4 Points d'attention

1 Le système cible

Un `.exe` se construit sous Windows, un binaire macOS sous macOS : pas de commande unique.

2 Config et données à côté

L'exécutable contient le code, pas `config.yaml` ni les fichiers de données.

3 Des fichiers lourds

Plusieurs dizaines de Mo dès qu'on utilise pandas.

4 Des alertes de sécurité

Un exécutable non signé peut déclencher SmartScreen, Gatekeeper ou l'antivirus.

Pour des utilisateurs qui ont uv : `uv tool install .` installe la commande, sans exécutable à construire.

TP bonus B : empaqueter le script en exécutable

Dans le dossier exercices/tpbonus-b-executable, à partir du TP5

1

Construire

Un script de lancement, puis PyInstaller en `--onefile` et en `--onedir`.

2

Comparer

Taille et temps de démarrage des deux modes.

3

Livrer

Un dossier `distribution/` avec l'exécutable et sa configuration, testé sans Python ni uv.

05

De belles lignes de commande

Rich | Typer

5 Rich : couleurs, tableaux, progression

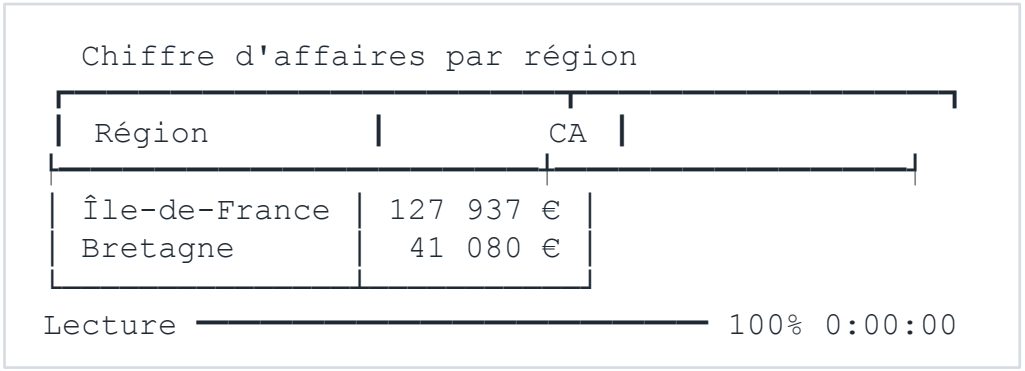
```
from rich.console import Console
from rich.logging import RichHandler
from rich.progress import track
from rich.table import Table

table = Table(title="Chiffre d'affaires par région")
table.add_column("Région")
table.add_column("CA", justify="right")
table.add_row("Île-de-France", "127 937 €")
table.add_row("Bretagne", "41 080 €")
Console().print(table)

for fichier in track(fichiers, description="Lecture"):
    ...

logging.basicConfig(level="INFO", format="%(message)s",
                    handlers=[RichHandler()])
```

Résultat



`RichHandler` rend les journaux lisibles, sans changer le reste du code de journalisation.

5 Typer : une commande à partir des annotations

```
from typing import Annotated
import typer

app = typer.Typer(help="Outils de traitement des ventes.")

@app.command()
def indicateurs(
    config: Annotated[Path, typer.Option("--config", "-c")]
        = Path("config.yaml"),
    debut: Annotated[str | None, typer.Option(
        help="premier mois, AAAA-MM")] = None,
) -> None:
    """Calcule les indicateurs par mois et par région."""

@app.command()
def rapport(sortie: Path = Path("outputs/rapport.html")):
    """Produit le rapport HTML."""

if __name__ == "__main__":
    app()
```

Chaque fonction décorée devient une **sous-commande**

- Sa docstring devient son aide
- Ses paramètres deviennent des options, avec leur type et leur valeur par défaut
- Une aide mise en forme par Rich

```
uv run python outils.py --help
uv run python outils.py indicateurs --debut 2025-07
```

06

Autres notions de Python intermédiaire

dataclasses | Décorateurs | with | itertools et functools

6 Les dataclasses

```
from dataclasses import asdict, dataclass, field

@dataclass
class Indicateur:
    region: str
    ca: float
    objectif: float
    alertes: list[str] = field(default_factory=list)

    @property
    def taux(self) -> float:
        return self.ca / self.objectif

bretagne = Indicateur("Bretagne", 6201.0, 4500)
bretagne.taux     # 1.378
asdict(bretagne)  # un dictionnaire
```

Constructeur, affichage et comparaison **générés** à partir des annotations

`bretagne.ca` plutôt que `bretagne["ca"]` : l'éditeur et mypy connaissent les champs

Valeur par défaut mutable : `field(default_factory=list)`

6 Les décorateurs

```
import functools, time

def chronometrer(fonction):
    @functools.wraps(fonction)
    def enveloppe(*args, **kwargs):
        debut = time.perf_counter()
        resultat = fonction(*args, **kwargs)
        duree = time.perf_counter() - debut
        print(f"{fonction.__name__} : {duree:.3f} s")
        return resultat
    return enveloppe

@chronometrer
def charger_ventes(dossier):
    ...
```

- Une fonction qui prend une fonction et en renvoie une version enrichie

`@chronometrer` équivaut à `charger_ventes = chronometrer(charger_ventes)`
`functools.wraps` garde le nom et la documentation
Déjà utilisés : `@pytest.fixture`, `@st.cache_data`, `@app.command()`

6 Les gestionnaires de contexte

with, depuis le premier jour

```
with open("rapport.txt", "w",
          encoding="utf-8") as f:
    f.write("...")
# fichier fermé ici, quoi qu'il arrive
```

L'équivalent d'un `try` dont le `finally` libère la ressource, même en cas d'erreur.

Écrire le sien avec contextlib

```
from contextlib import contextmanager

@contextmanager
def chrono(etape: str):
    debut = time.perf_counter()
    try:
        yield
    finally:
        duree = time.perf_counter() - debut
        print(f"{etape} : {duree:.3f} s")

with chrono("chargement"):
    ventes = charger_ventes(dossier)
```

6 itertools.groupby : trier d'abord

```
from itertools import groupby

ventes = [("IDF", 120), ("IDF", 80), ("BRE", 50), ("IDF", 30)]

[(r, sum(m for _, m in g)) for r, g in groupby(ventes, key=lambda v: v[0])]
# [('IDF', 200), ('BRE', 50), ('IDF', 30)]      l'Île-de-France apparaît deux fois

[(r, sum(m for _, m in g)) for r, g in groupby(sorted(ventes), key=lambda v: v[0])]
# [('BRE', 50), ('IDF', 230)]
```

`itertools.groupby` regroupe les éléments **consécutifs**. Le `groupby` de pandas n'a pas cette contrainte.

6 functools : reduce et partial

reduce : de proche en proche

```
from functools import reduce

tables = [ventes, details, stocks]
reduce(
    lambda g, d: g.merge(d, on="id_produit"),
    tables,
)
```

Joindre une liste de tables sur la même clé, quelle que soit sa longueur.

partial : fixer des arguments

```
from functools import partial

lire_export = partial(
    pd.read_csv,
    sep=";", decimal=",", encoding="cp1252",
)
lire_export("export_janvier.csv")
```

Une fonction de lecture des exports Excel français, réglée une fois pour toutes.

À retenir

- Polars et DuckDB : plusieurs fois plus rapides sur les gros volumes

Polars : expressions, `scan_*` et `collect()`, `how="diagonal"`
DuckDB : SQL sur des fichiers, `union_by_name = true`
Streamlit : script réexécuté, `@st.cache_data`

- tkinter pour un utilitaire, Qt pour une vraie interface

PyInstaller : un exécutable par système, `--onedir` démarre vite

- Rich pour la sortie, Typer pour la commande
- dataclasses pour regrouper des données

Décorateurs et `with` pour enrichir et sécuriser
`itertools.groupby` exige des données triées